

Lecture 11: RNNs

Instructor: Stephanie Schoch

CSCI 680: Natural Language Processing
Department of Computer Science, College of William & Mary
September 30, 2026



Lecture Outline

- RNNs
- Applications
- Training RNNs / BPTT
 - Vanishing gradients
- LSTMs & Bidirectional RNNs

Recurrent Neural Networks (RNNs)

Modeling Time in Neural Networks

- Language is inherently temporal

star trek was kind of **terrific** once , **but now** it is a copy of a copy of a copy .

0 (negative)

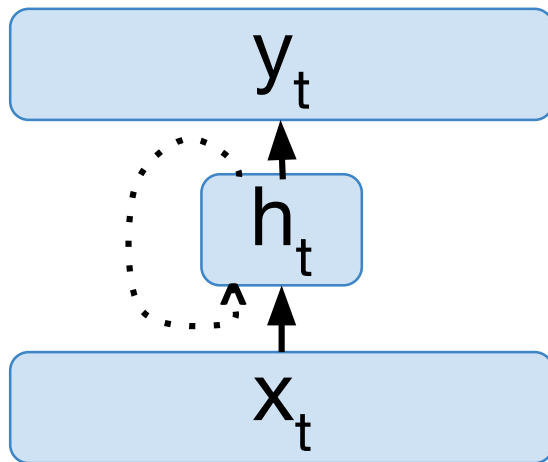
idx 44,792

Later words can change how earlier words should be interpreted.

- Yet the simple NLP classifiers we've seen (for example for sentiment analysis) mostly ignore time
 - FFNN-LMs use a “moving window” approach to time
- **Today:** RNNs and their variants like LSTMs (that have a different way of representing time!)

Recurrent Neural Networks

- **RNN:** Any network that contains a cycle within its network connections.
 - Processes a sequence one element at a time.
- At each step, the hidden state h_t is computed from the current input x_t and previous hidden state h_{t-1}
 - The activation value h_t depends on x_t but also h_{t-1}
 - Dependent on its own earlier outputs as input (recurrence)
- RNNs compared to n -gram and FF-LMs:
 - Don't have limited context problem of n -grams or fixed context of FF-LMs.
 - The hidden state can *in principle* encode information from the entire preceding sequence.



RNN Language Model

Output Distribution:

$$\hat{y}^{(t)} = \text{softmax}(Uh^{(t)} + b_2) \in \mathbb{R}^{|V|}$$

Hidden States:

$$h^{(t)} = \sigma(W_h h^{(t-1)} + W_e e^{(t)} + b_1)$$

$h^{(0)}$ is the initial hidden state

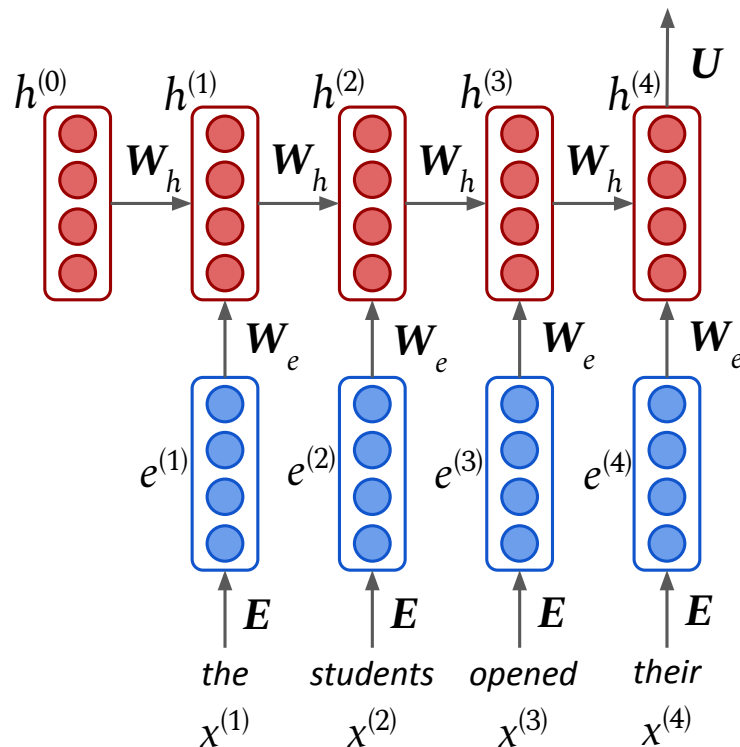
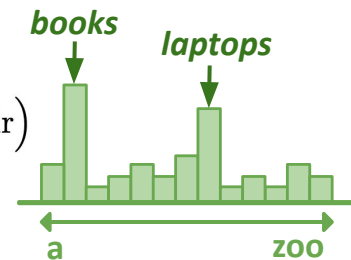
Word Embeddings:

$$e^{(t)} = Ex^{(t)}$$

Words / One-Hot Vectors:

$$x^{(t)} \in \mathbb{R}^{|V|}$$

$$\hat{y}^{(4)} = P(x^{(5)} \mid \text{the students opened their})$$



RNN Language Model

$$e^{(t)} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \quad h^{(t-1)} = \begin{bmatrix} 3 \\ 1 \end{bmatrix}, \quad b_1 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

$$W_e = \begin{bmatrix} 1 & 0 \\ 0 & 2 \end{bmatrix}, \quad W_h = \begin{bmatrix} 0.5 & 1 \\ 1 & 0 \end{bmatrix}$$

$$\underbrace{h^{(t)}}_{d_h \times 1} = \sigma \left(\underbrace{W_h}_{d_h \times d_h} \underbrace{h^{(t-1)}}_{d_h \times 1} + \underbrace{W_e}_{d_h \times d_e} \underbrace{e^{(t)}}_{d_e \times 1} + \underbrace{b_1}_{d_h \times 1} \right)$$

RNN Language Model

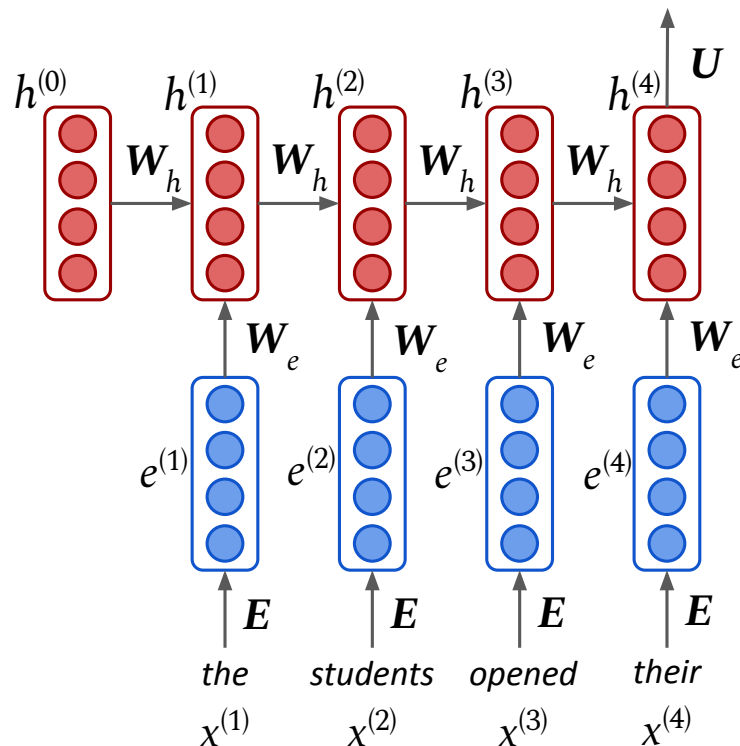
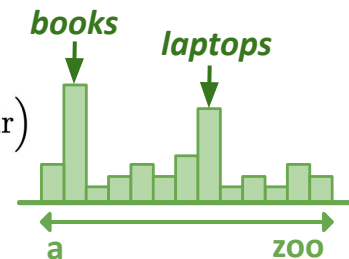
RNN Advantages:

- Can process **any length** input
- Computation for step t
- can (in theory) use information from **many steps back**
- **Model size doesn't increase** for longer input
- Weights are shared across timesteps, so each position is processed in the **same way**.

RNN Disadvantages:

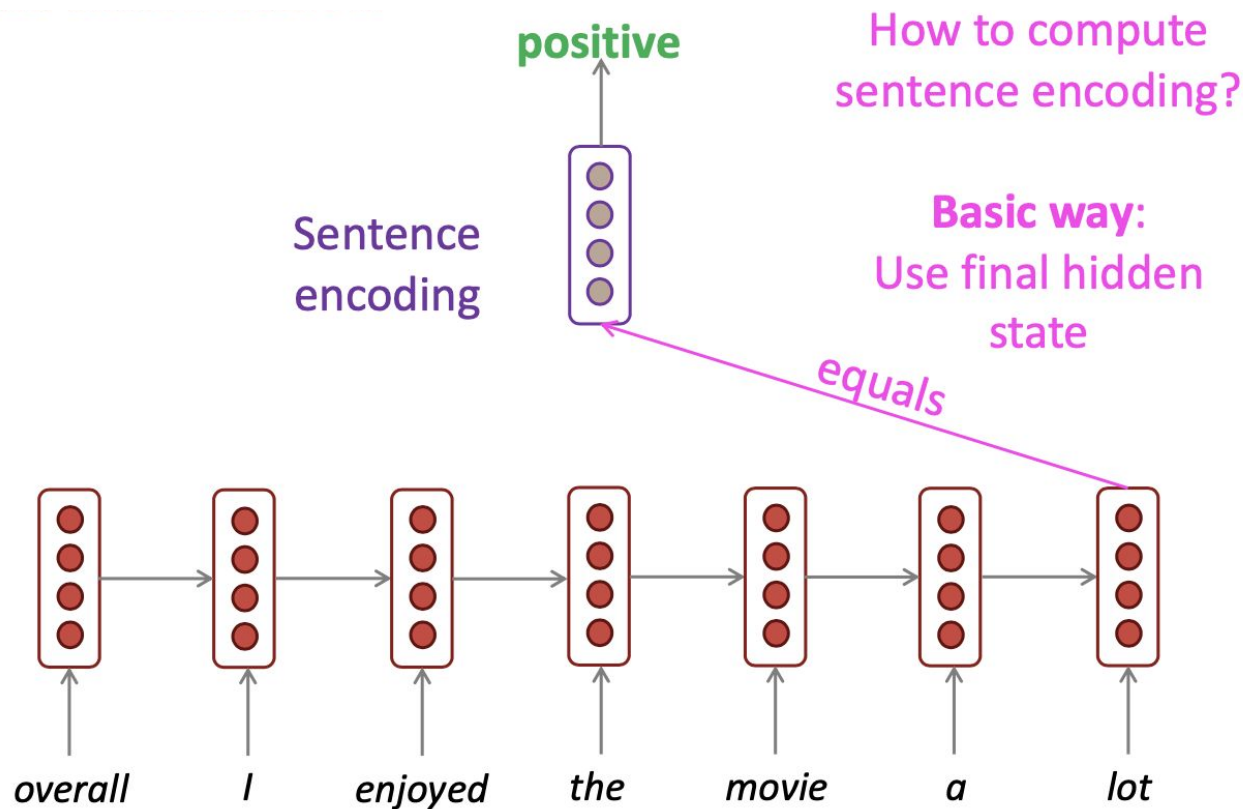
- Recurrent computation is **slow**
- In practice, difficult to access information from **many steps back**

$$\hat{y}^{(4)} = P(x^{(5)} \mid \text{the students opened their})$$

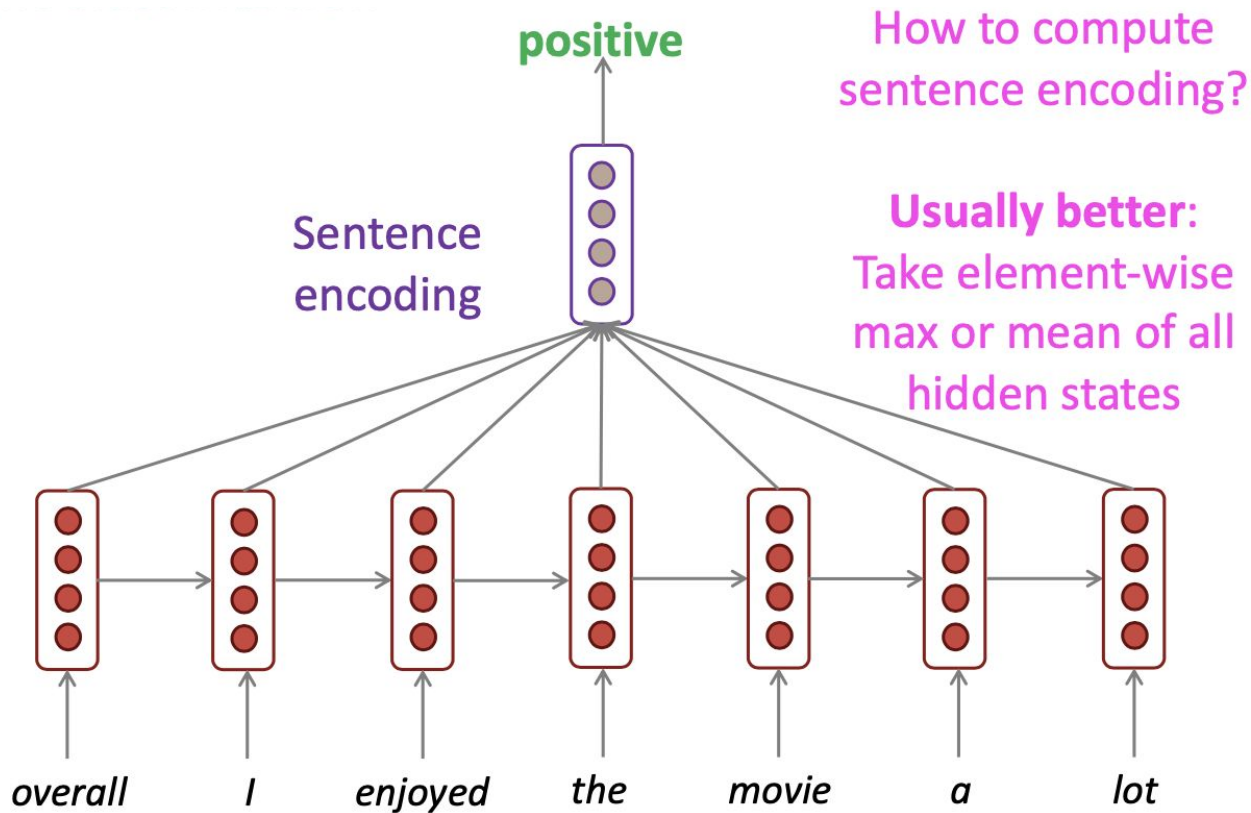


Applications

Sentence Classification

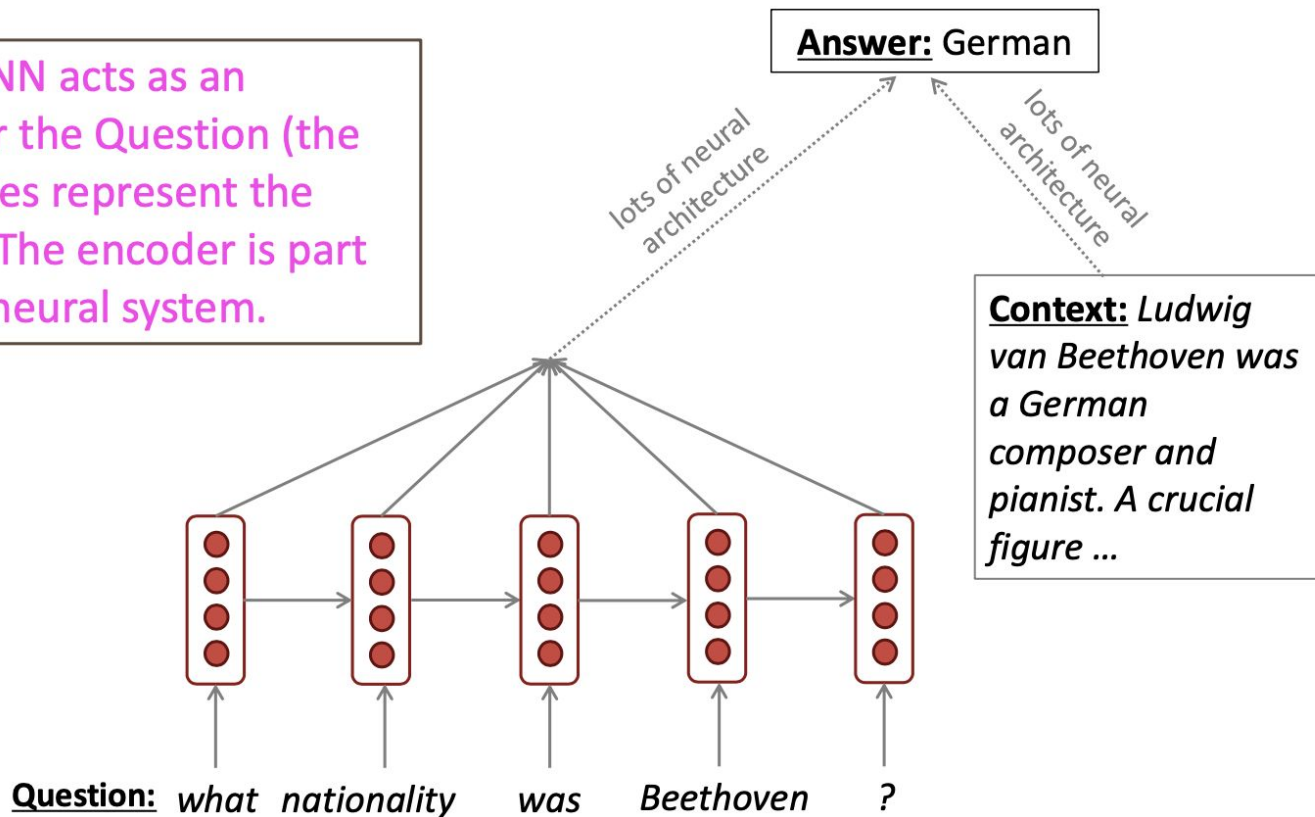


Sentence Classification



Language Encoding Module

Here the RNN acts as an **encoder** for the Question (the hidden states represent the Question). The encoder is part of a larger neural system.



Training Recurrent Neural Networks (RNNs)

Training a RNN Language Model

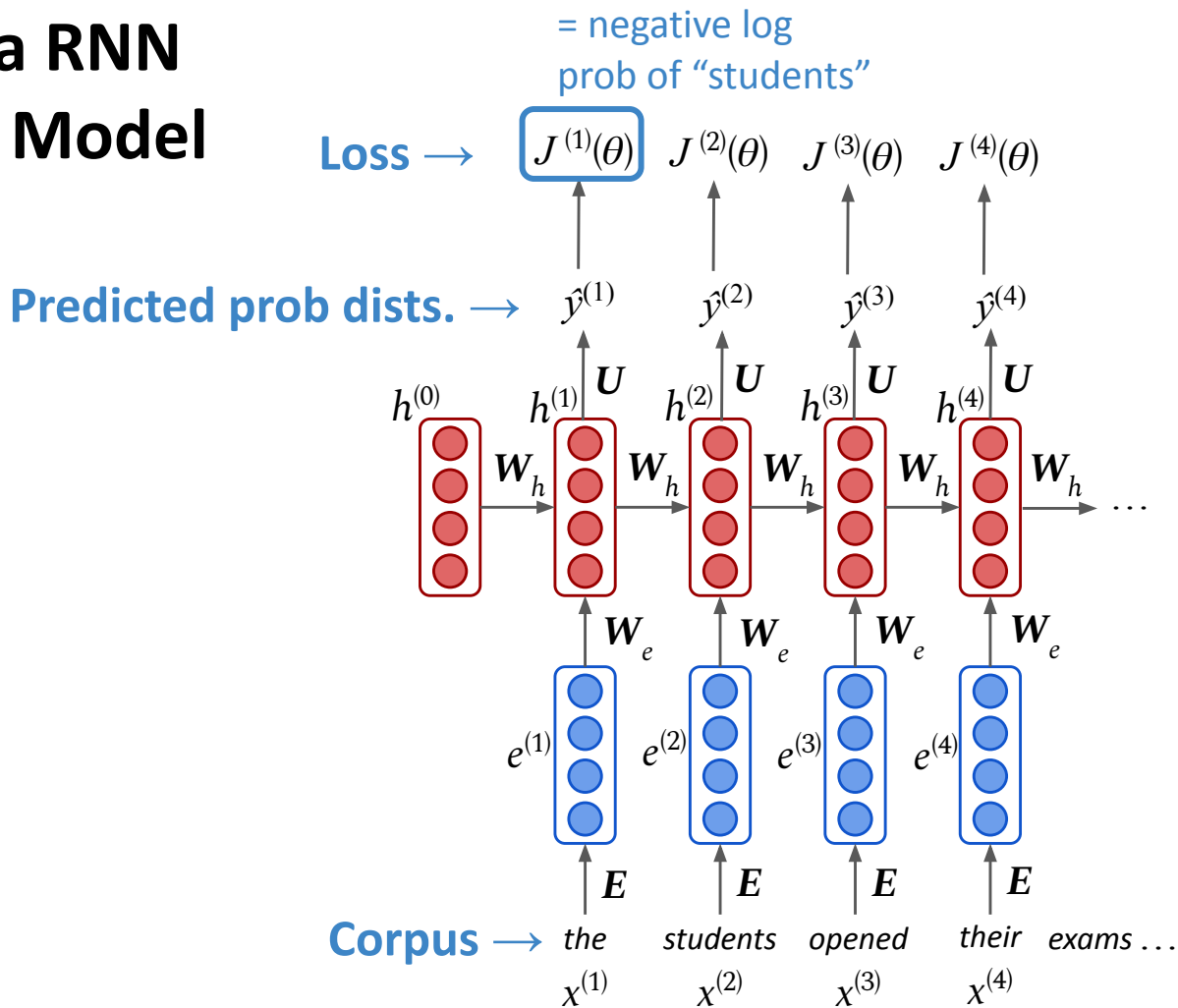
- Get a **big corpus of text**, which is a sequence of words $x^{(1)}, x^{(2)}, \dots, x^{(T)}$
- Feed into RNN-LM; compute output distribution $\hat{y}^{(t)}$ **for every step t** .
 - i.e. predict probability dist of *every word*, given words so far
- Loss function on step t is **cross-entropy** between predicted probability distribution $\hat{y}^{(t)}$, and the true next word $y^{(t)}$ (one-hot for $x^{(t+1)}$):

$$J^{(t)}(\theta) = CE(y^{(t)}, \hat{y}^{(t)}) = - \sum_{w \in V} y_w^{(t)} \log \hat{y}_w^{(t)} = - \log \hat{y}_{x_{t+1}}^{(t)}$$

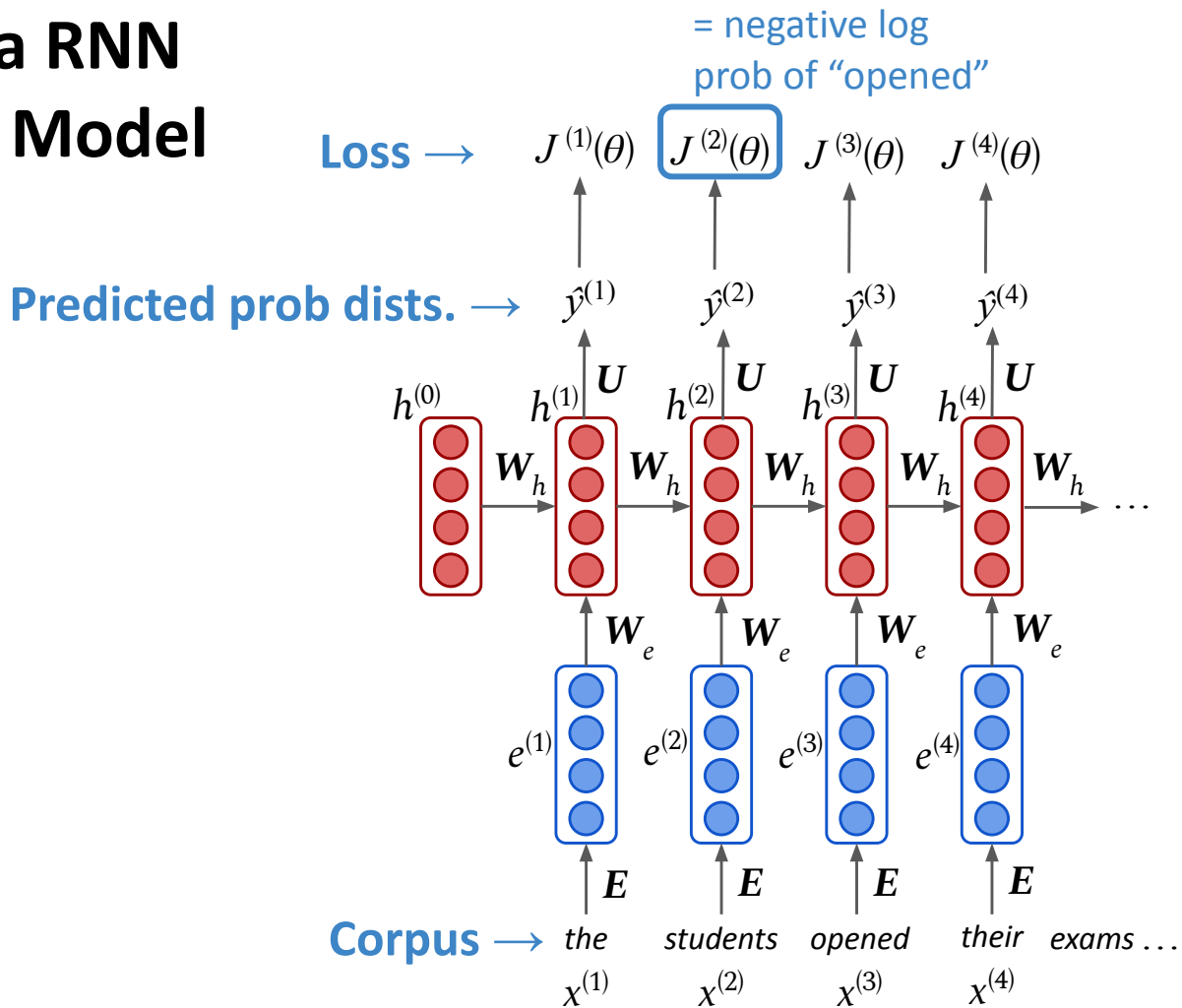
- Average this to get **overall loss** for entire training set:

$$J(\theta) = \frac{1}{T} \sum_{t=1}^T J^{(t)}(\theta) = \frac{1}{T} \sum_{t=1}^T - \log \hat{y}_{x_{t+1}}^{(t)}$$

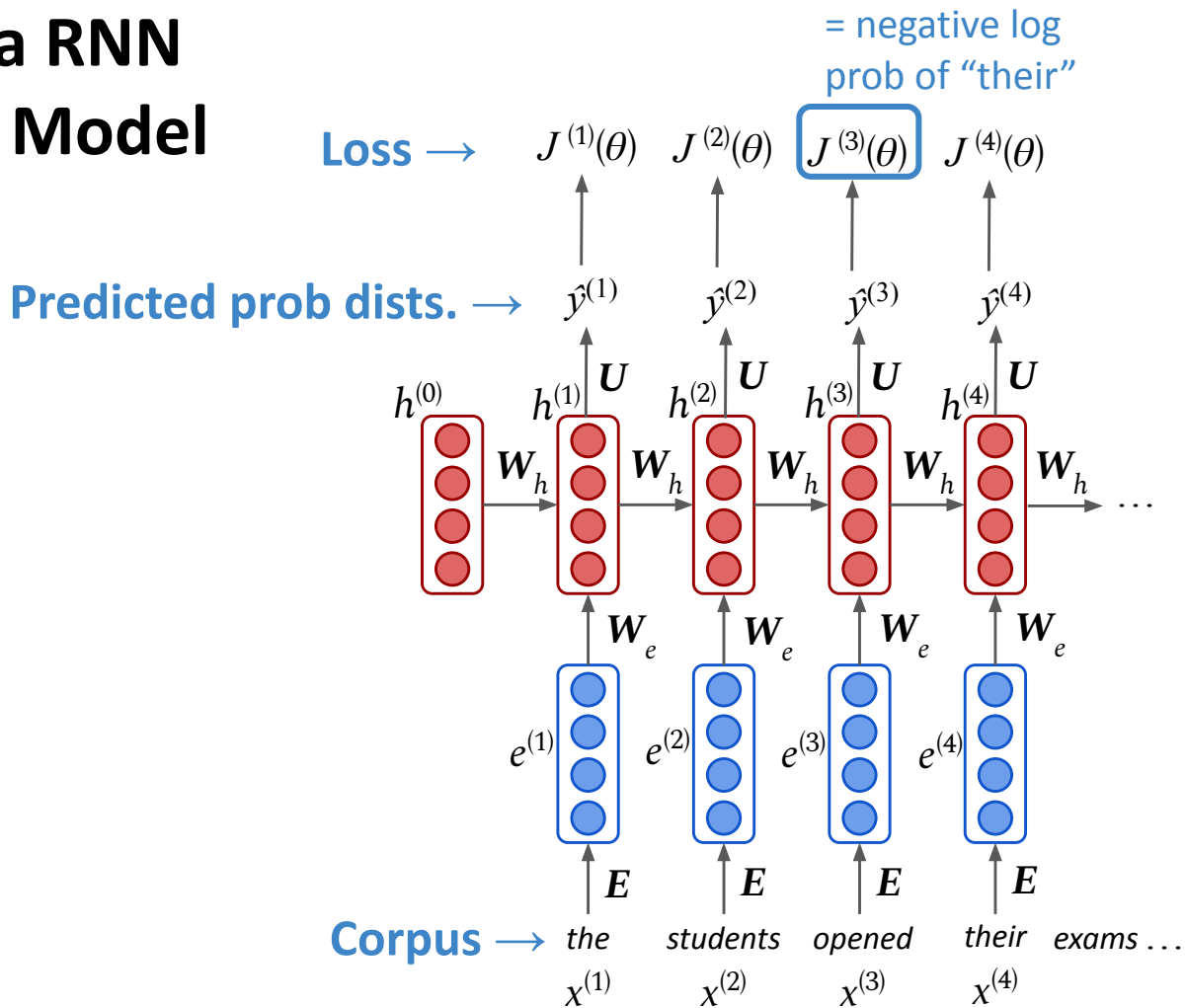
Training a RNN Language Model



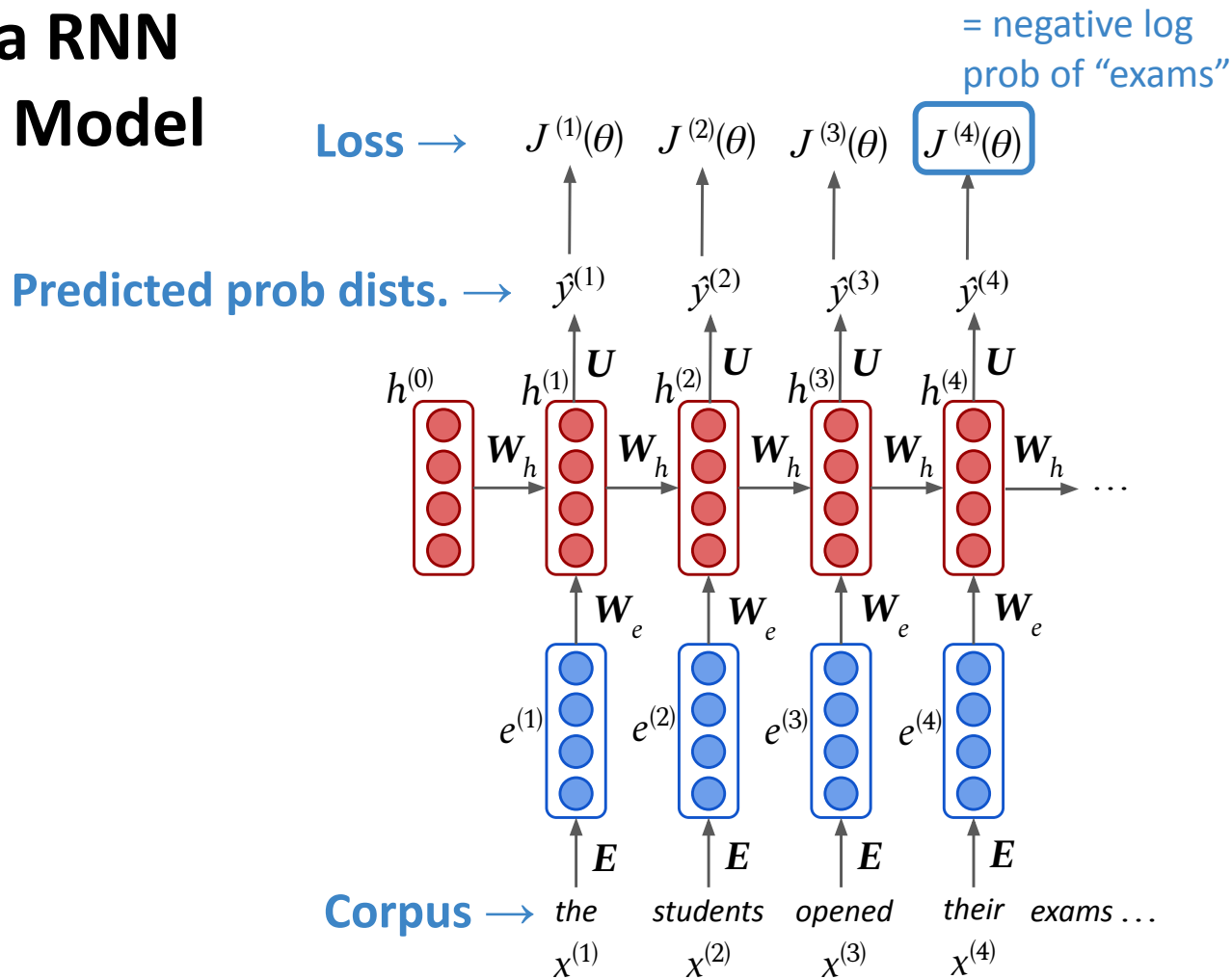
Training a RNN Language Model



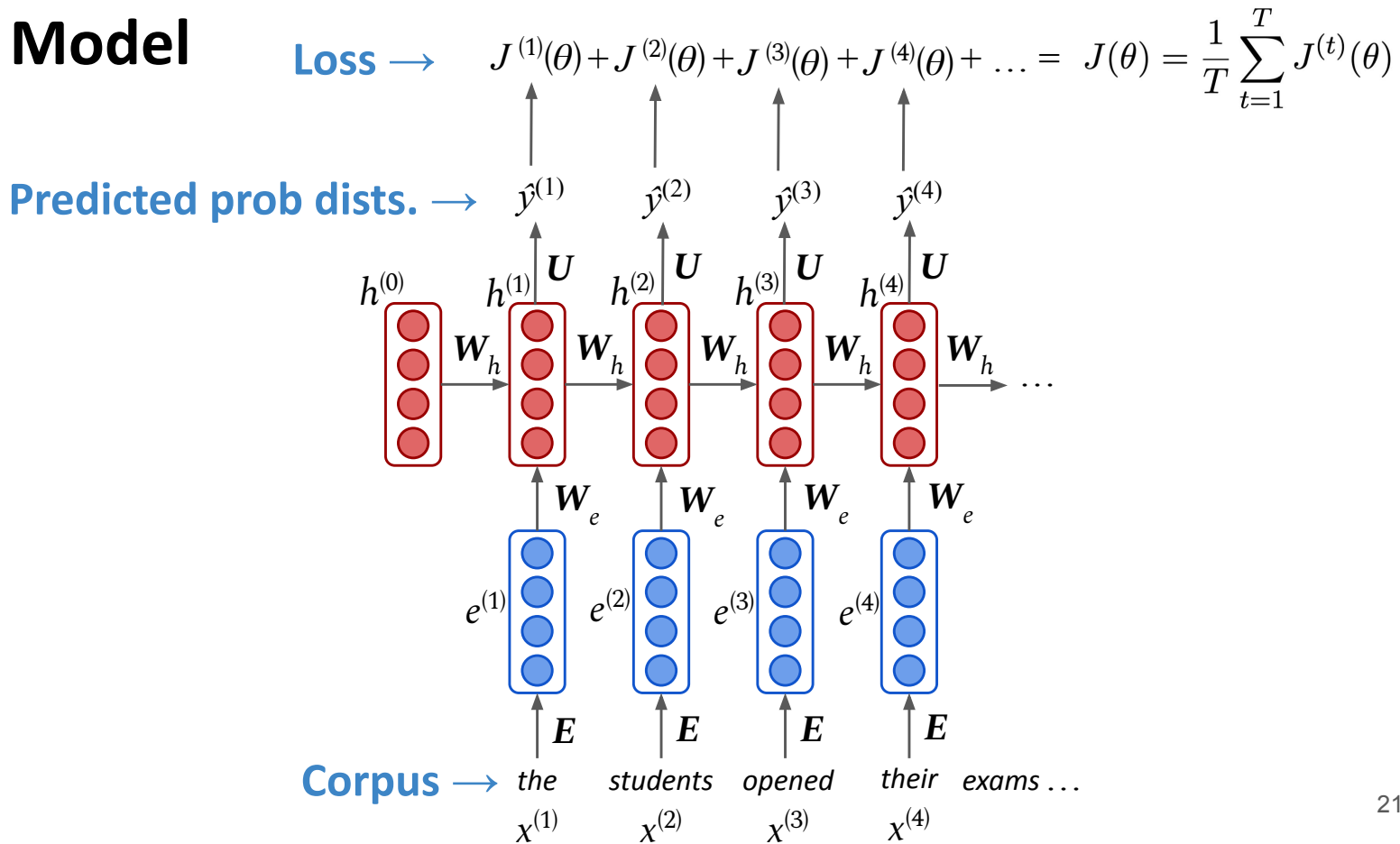
Training a RNN Language Model



Training a RNN Language Model



Training a RNN Language Model



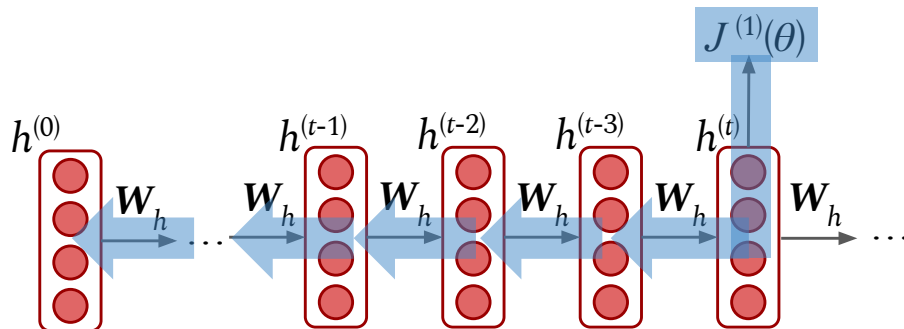
Training a RNN Language Model

- However: Computing loss and gradients across entire corpus $x^{(1)}, x^{(2)}, \dots, x^{(T)}$ is **too expensive**.

$$J(\theta) = \frac{1}{T} \sum_{t=1}^T J^{(t)}(\theta)$$

- In practice, consider $x^{(1)}, x^{(2)}, \dots, x^{(T)}$ as a **sentence** (or a **document**)
- Recall: **Stochastic Gradient Descent** allows us to compute loss
- and gradients for small chunk of data, and update.
- Compute loss $J(\theta)$ for a sentence (actually, a batch of sentences), compute gradients and update weights. Repeat.

Computing the Gradients: Backpropagation Through Time



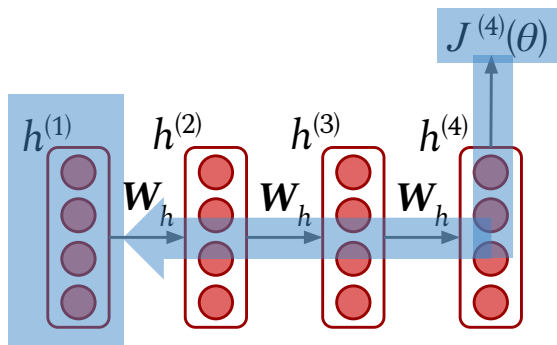
- Derivative of $J(\theta)$ w.r.t. the repeated weight matrix W_h
 - The gradient w.r.t. a repeated weight is the sum of the gradient w.r.t. each time it appears.

$$\frac{\partial J^{(t)}}{\partial W_h} = \sum_{i=1}^t \left. \frac{\partial J^{(t)}}{\partial W_h} \right|_{(i)}$$



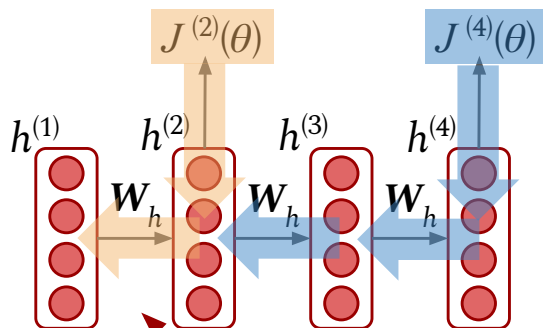
Backpropagation through time: Backpropagate over timesteps $i=t, \dots, 0$, summing gradients as you go.

Computing the Gradients: Vanishing Gradient Problem



$$\frac{\partial J^{(4)}}{\partial h^{(1)}} = ?$$

Why are Vanishing Gradients a Problem?



Gradient signal from far away is lost because it's much smaller than gradient signal from close-by. So, model weights are updated only with respect to near effects, not long-term effects.

Effect of Vanishing Gradient on RNN-LM

- **LM task:** *When she tried to print her tickets, she found that the printer was out of toner. She went to the stationery store to buy more toner. It was very overpriced. After installing the toner into the printer, she finally printed her _____*
- To learn from this training example, the RNN-LM needs to **model the dependency** between “*tickets*” on the 7th step and the target word “*tickets*” at the end.
- But if gradient is small, the model **can't learn this dependency**
 - So, the model is **unable to predict similar long-distance dependencies** at test time

Long Short-Term Memory RNNs (LSTMs) & Bidirectional RNNs

How to fix the vanishing gradient problem?

- The main problem is that *it's too difficult for the RNN to learn to preserve information over many timesteps.*
- In a vanilla RNN, the hidden state is constantly being *rewritten*

$$h^{(t)} = \sigma\left(W_h h^{(t-1)} + W_x x^{(t)} + b\right)$$

New design: Equip an RNN with separate memory data structure

Long Short-Term Memory RNNs (LSTMs)

A type of RNN proposed by Hochreiter and Schmidhuber in 1997 as a solution to the vanishing gradients problem (another important variant: Gersetal.(2000))

- On step t , there is a **hidden state** $h^{(t)}$ and a **cell state** $c^{(t)}$
 - Both are vectors of length n
 - The cell state stores **long-term information**
- The LSTM can **read**, **erase**, and **write** information from the cell
 - The cell becomes conceptually rather like RAM in a computer
- The selection of which information is erased/written/read is controlled by three corresponding **gates**
 - The gates are also vectors length n
- On each timestep, each element of the gates can be **open** (1), **closed** (0), or somewhere in-between
- The gates are **dynamic**: their value is computed based on the current context

Long Short-Term Memory (LSTM)

We have a sequence of inputs $x^{(t)}$, and we will compute a sequence of hidden states $h^{(t)}$ and cell states $c^{(t)}$. On timestep t :

Forget gate: controls what is kept vs forgotten, from previous cell state

Input gate: controls what parts of the new cell content are written to cell

Output gate: controls what parts of cell are output to hidden state

New cell content: this is the new content to be written to the cell

Cell state: erase (“forget”) some content from last cell state, and write (“input”) some new cell content

Hidden state: read (“output”) some content from the cell

Sigmoid function: all gate values are between 0 and 1

$$\begin{aligned} f^{(t)} &= \sigma \left(W_f h^{(t-1)} + U_f x^{(t)} + b_f \right) \\ i^{(t)} &= \sigma \left(W_i h^{(t-1)} + U_i x^{(t)} + b_i \right) \\ o^{(t)} &= \sigma \left(W_o h^{(t-1)} + U_o x^{(t)} + b_o \right) \end{aligned}$$

$$\begin{aligned} \tilde{c}^{(t)} &= \tanh \left(W_c h^{(t-1)} + U_c x^{(t)} + b_c \right) \\ c^{(t)} &= f^{(t)} \circ c^{(t-1)} + i^{(t)} \circ \tilde{c}^{(t)} \\ h^{(t)} &= o^{(t)} \circ \tanh c^{(t)} \end{aligned}$$

All these are vectors of same length n

Gates are applied using element-wise (or Hadamard) product: $\odot$

Long Short-Term Memory (LSTM): Intuitive Example

The cats in the house are...

Long Short-Term Memory (LSTM): Intuitive Example

The cats in the house are...

Forget gate $f^{(t)}$: “How much of the old memory should I keep?”

Near 1: keep this **Near 0:** get rid of this
e.g. while reading “in the house,” keep the
memory that the subject “cats” is plural

Long Short-Term Memory (LSTM): Intuitive Example

The cats in the house are...

Forget gate $f^{(t)}$: “How much of the old memory should I keep?”

Near 1: keep this **Near 0:** get rid of this
e.g. while reading “in the house,” keep the memory that the subject “cats” is plural

Input gate $i^{(t)}$: “How much new information should I write into memory?”

Near 1: allow new **Near 0:** block new
e.g. at “cats,” write subject = plural; at “house,” don’t overwrite that subject-number memory

Long Short-Term Memory (LSTM): Intuitive Example

The cats in the house are...

Forget gate $f^{(t)}$: “How much of the old memory should I keep?”

Near 1: keep this **Near 0:** get rid of this
e.g. while reading “in the house,” keep the memory that the subject “cats” is plural

Input gate $i^{(t)}$: “How much new information should I write into memory?”

Near 1: allow new **Near 0:** block new
e.g. at “cats,” write subject = plural; at “house,” don’t overwrite that subject-number memory

Candidate cell content $\tilde{c}^{(t)}$: “What new information might be useful to store?”

Large +/-: propose new info **Near 0:** propose little info
e.g. at “cats,” propose a representation corresponding to plural subject

Long Short-Term Memory (LSTM): Intuitive Example

The cats in the house are...

Forget gate $f^{(t)}$: “How much of the old memory should I keep?”

Near 1: keep this **Near 0:** get rid of this
e.g. while reading “in the house,” keep the memory that the subject “cats” is plural

Input gate $i^{(t)}$: “How much new information should I write into memory?”

Near 1: allow new **Near 0:** block new
e.g. at “cats,” write subject = plural; at “house,” don’t overwrite that subject-number memory

Candidate cell content $\tilde{c}^{(t)}$: “What new information might be useful to store?”

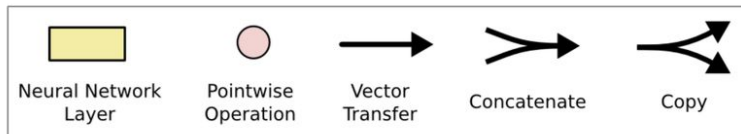
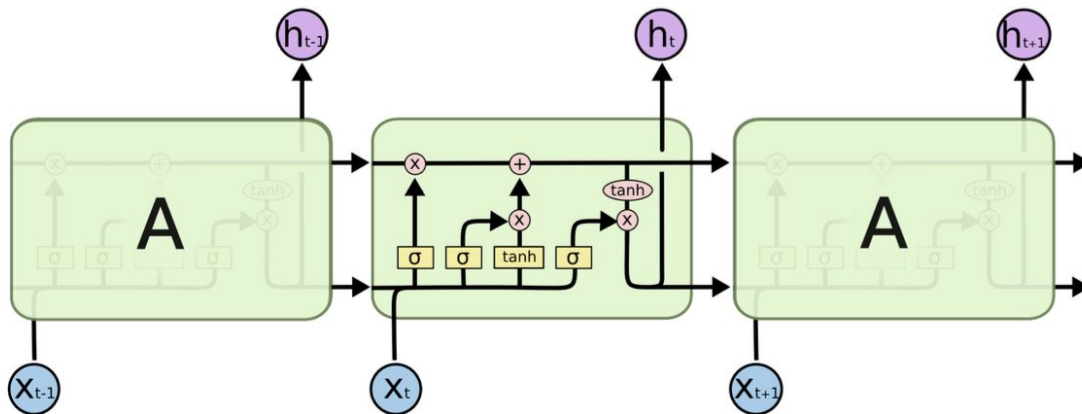
Large +/-: propose new info **Near 0:** propose little info
e.g. at “cats,” propose a representation corresponding to plural subject

Output gate $o^{(t)}$: “How much of the memory should I expose right now?”

Near 1: expose this memory **Near 0:** don’t expose
e.g. “After reading ‘house,’ expose the stored plurality information so it can help predict ‘are.’”

Long Short-Term Memory (LSTM)

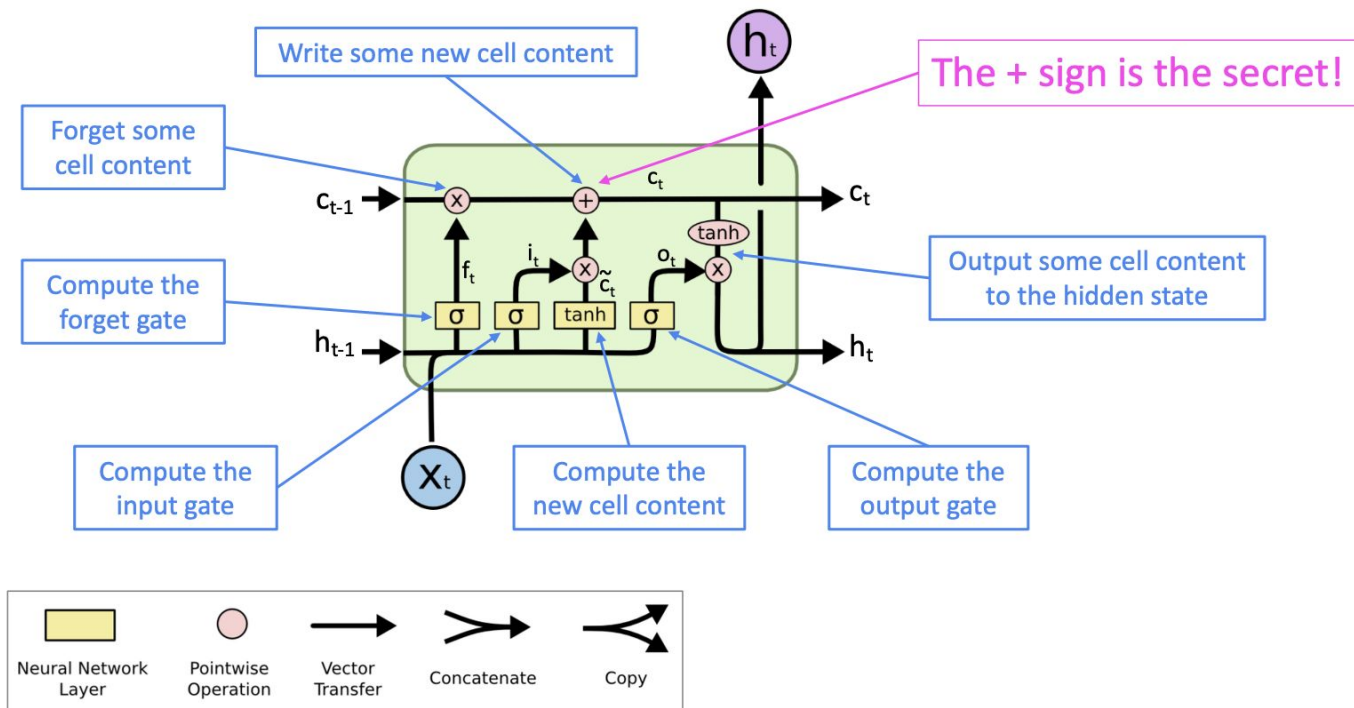
You can think of the LSTM equations visually like this:



Source: <http://colah.github.io/posts/2015-08-Understanding-LSTMs/>

Long Short-Term Memory (LSTM)

You can think of the LSTM equations visually like this:



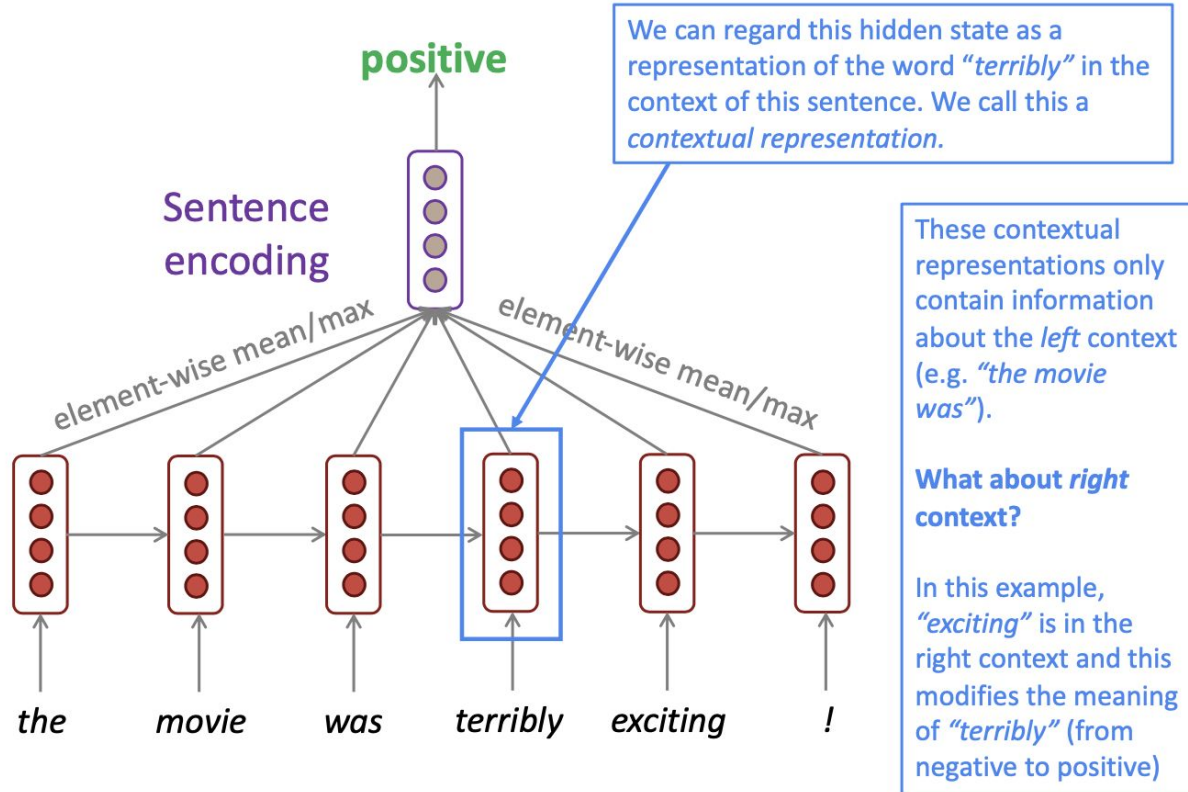
Source: <http://colah.github.io/posts/2015-08-Understanding-LSTMs/>

How does LSTM solve vanishing gradients?

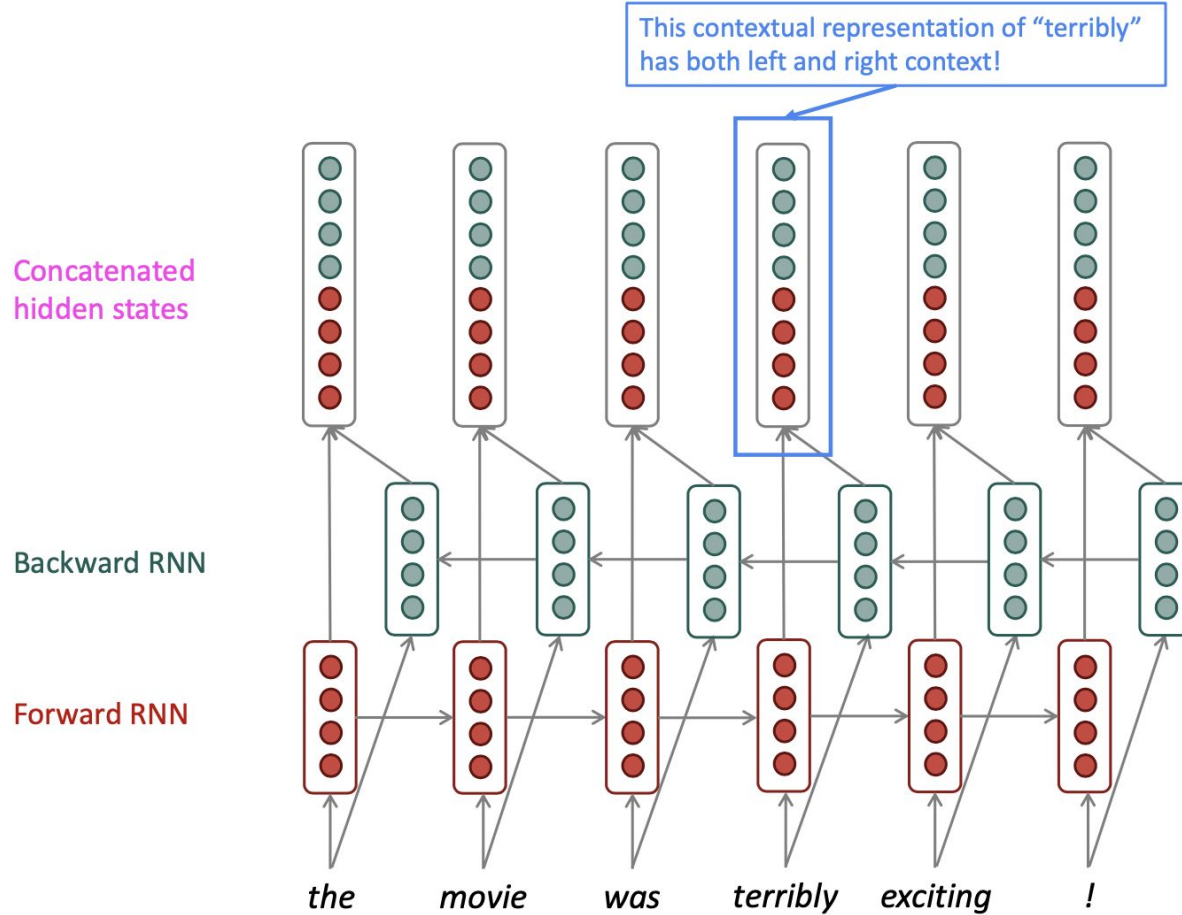
- The LSTM architecture makes it easier for the RNN to **preserve information over many timesteps**
 - e.g., if the forget gate is set to 1 for a cell dimension and the input gate set to 0, then the information of that cell is preserved indefinitely.
 - In contrast, it's harder for a vanilla RNN to learn a recurrent weight matrix W_h that preserves info in the hidden state
 - In practice, you get about 100 timesteps rather than about 7
- LSTM doesn't guarantee that there is no vanishing (or exploding) gradient, but it does provide an easier way for the model to learn long-distance dependencies

Bidirectional RNNs

Task: Sentiment Classification



Bidirectional RNNs



Bidirectional RNNs

On timestep t :

This is a general notation to mean “compute one forward step of the RNN” – it could be a vanilla, LSTM or GRU computation.

Forward RNN $\vec{h}^{(t)} = \text{RNN}_{\text{FW}}(\vec{h}^{(t-1)}, \mathbf{x}^{(t)})$

Backward RNN $\overleftarrow{h}^{(t)} = \text{RNN}_{\text{BW}}(\overleftarrow{h}^{(t+1)}, \mathbf{x}^{(t)})$

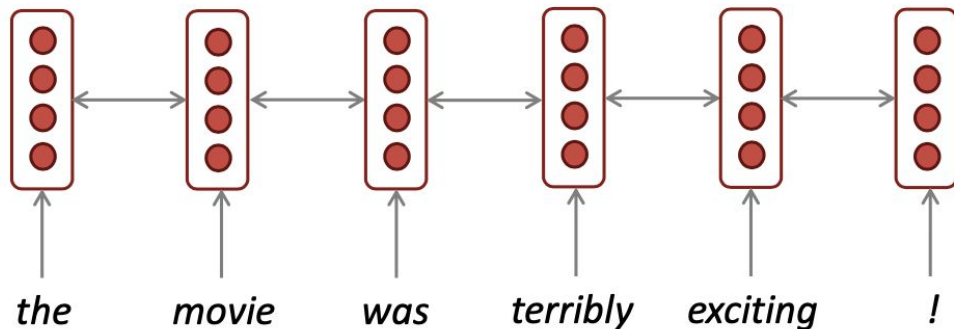
} Generally, these two RNNs have separate weights

Concatenated hidden states $\mathbf{h}^{(t)} = [\vec{h}^{(t)}; \overleftarrow{h}^{(t)}]$

We regard this as “the hidden state” of a bidirectional RNN. This is what we pass on to the next parts of the network.

Bidirectional RNNs

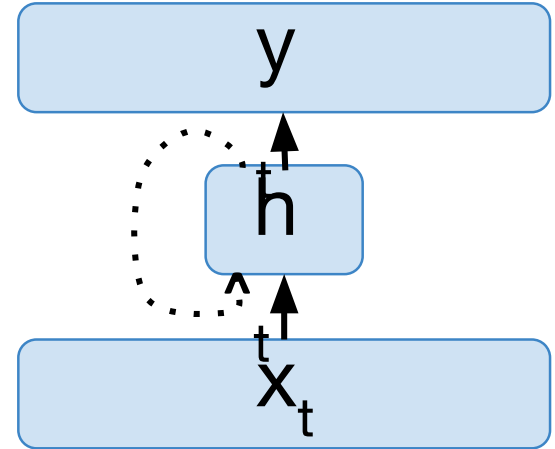
- Note: bidirectional RNNs are only applicable if you have access to the entire input sequence
- They are not applicable to Language Modeling, because in LM you only have left context available.
- If you do have entire input sequence (e.g., any kind of encoding), bidirectionality is powerful (you should use it by default).



The two-way arrows indicate bidirectionality and the depicted hidden states are assumed to be the concatenated forwards+backwards states

Summary: RNNs

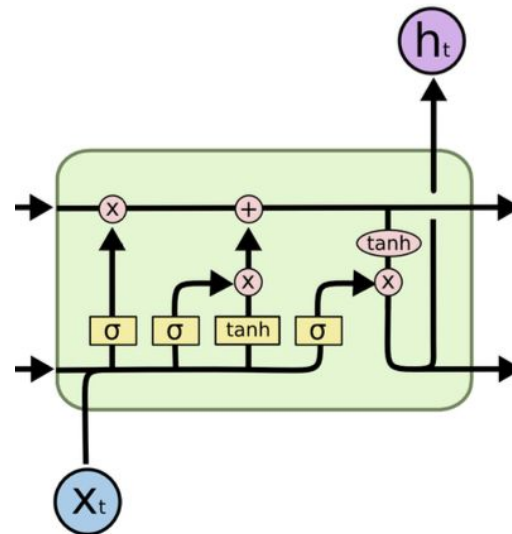
- Recurrent Neural Networks processes sequences one element at a time
- RNNs do not have:
 - the limited context problem of n-gram models
 - the fixed context limitation of feedforward LMs
 - since the hidden state can in principle represent information about all of the preceding words all the way back to the beginning of the sequence
- But training RNNs is hard:
 - Vanishing gradient problem
 - LSTMs address it by incorporating a memory



Summary: LSTMs

- The LSTM architecture makes it much easier for an RNN to preserve information over many timesteps
 - e.g., if the forget gate is set to 1 for a cell dimension and the input gate set to 0, then the information of that cell is preserved indefinitely
- In 2013–2015, LSTMs started achieving state-of-the-art results

Successful tasks include handwriting recognition, speech recognition, machine translation, parsing, and image captioning, as well as language models
- LSTMs became the dominant approach for most NLP tasks



TODOs:

- **Homework 1 Due: 9/30**
- **Project Proposal Due: 10/02**

Next Class: Sequence-to-Sequence & Attention